HELWAN UNIVERSITY

Faculty of

Computer and Systems Engineering Department

4th Year — Academic Year 2025/2026

Cryptography Lib Lab

Path 3 · Hybrid Encryption · Secure Student Records

Course: Embeded Security / Cryptography Lab ·

Prepared By	**Ahmed Ehab Mahmoud** **Hassan Adel Hassan**
Supervised By	Eng. Abdelrhman Yasser El-Arabawy

PROJECT OVERVIEW

This project builds a real-data encryption and decryption application using Path 3: Hybrid Encryption. The scenario is Secure Student Records, where a CSV file containing 15 fake student records is protected using a two-layer cryptographic approach. AES-256-CBC encrypts the file content, and RSA-2048 (OAEP) encrypts the AES session key. A SHA-256 hash check proves the decrypted file is byte-for-byte identical to the original. The application ships with a dark-themed Tkinter GUI and a CLI fallback.

Property	Value
Scenario	`Secure Student Records — CSV file (15 records)`
Project Path	`Path 3: Hybrid Encryption`
Symmetric cipher	`AES-256-CBC (data encryption)`
Asymmetric cipher	`RSA-2048 with OAEP padding (key encryption)`
Integrity check	`SHA-256 hash comparison`
IV size	`128 bits (16 bytes) — randomly generated per run`
AES key size	`256 bits (32 bytes) — randomly generated per run`
Padding	`PKCS7`
Library	`PyCryptodome (Python 3)`
Interface	`Tkinter GUI + CLI menu`

SECURITY UNDERSTANDING — Q&A

Q1. Which algorithm did you choose, and why?

Path 3 (Hybrid Encryption) was chosen because it mirrors how real secure systems such as TLS and HTTPS work in practice. AES-256-CBC handles the actual file data because it is fast, efficient, and has no practical limit on data size. RSA-2048 handles key exchange because asymmetric encryption eliminates the need to share a secret key over an insecure channel. Using RSA alone to encrypt a file is not feasible because RSA-2048 can only encrypt a maximum of 245 bytes directly. The hybrid approach combines the speed of AES with the security of RSA into one practical system.

Q2. What is the key used for in your project?

Two keys are used. The **AES-256 session key** is a random 32-byte key generated fresh for every encryption run. It is fed into `AES.new(key, AES.MODE_CBC, iv)` to encrypt and decrypt the CSV file content. The **RSA-2048 key pair** handles key transport: the public key encrypts the AES session key so it can be stored safely in `metadata.json`, and the private key decrypts it at the start of every decryption operation.

Q3. What is the role of the IV in your project?

The Initialization Vector (IV) is a random 16-byte value generated fresh for every encryption using `get_random_bytes(16)`. In CBC mode it is XORed with the first plaintext block before encryption begins, which chains randomness into the entire ciphertext. This guarantees that encrypting the same file twice with the same AES key produces completely different ciphertext each time, preventing pattern analysis or replay attacks. The IV is not secret — it is saved openly in `metadata.json` alongside the encrypted AES key and is required for decryption.

Q4. What does the encrypted output look like compared to the original?

The original students.csv is human-readable text with names, student IDs, grades, and emails arranged in comma-separated rows. The encrypted output students_encrypted.bin is raw binary noise — when opened in a text editor it

displays meaningless symbols, and in a hex viewer it shows a stream of random-looking bytes with no visible structure. When displayed as Base64 it produces an unrecognisable string of characters. The file size increases only slightly (from 1097 to 1104 bytes) because PKCS7 padding rounds the final block to the nearest 16 bytes. No part of the original content is visible in the ciphertext.

Q5. How did you prove that decryption recovered the original data?

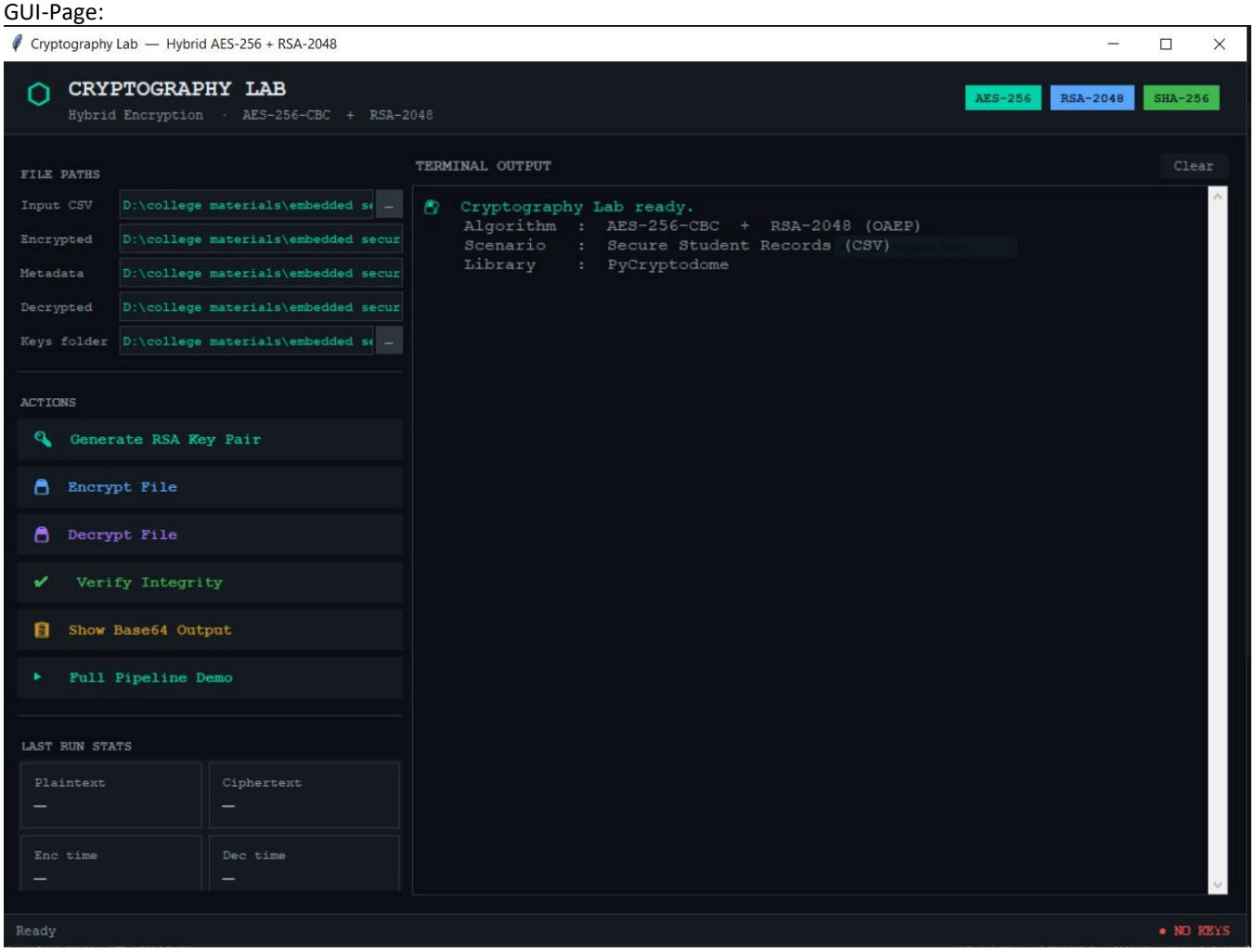
A SHA-256 hash is computed for both `students.csv` and `students_decrypted.csv` using the `verify_files()` function. SHA-256 produces a unique 64-character hex fingerprint for any file. If even a single byte differs between the two files, the hashes will be completely different. Both hashes are printed side-by-side in the terminal and the program outputs a clear **VERIFICATION SUCCESS** message when they match, confirming the decrypted file is byte-for-byte identical to the original.

Q6. What is one security limitation of your project?

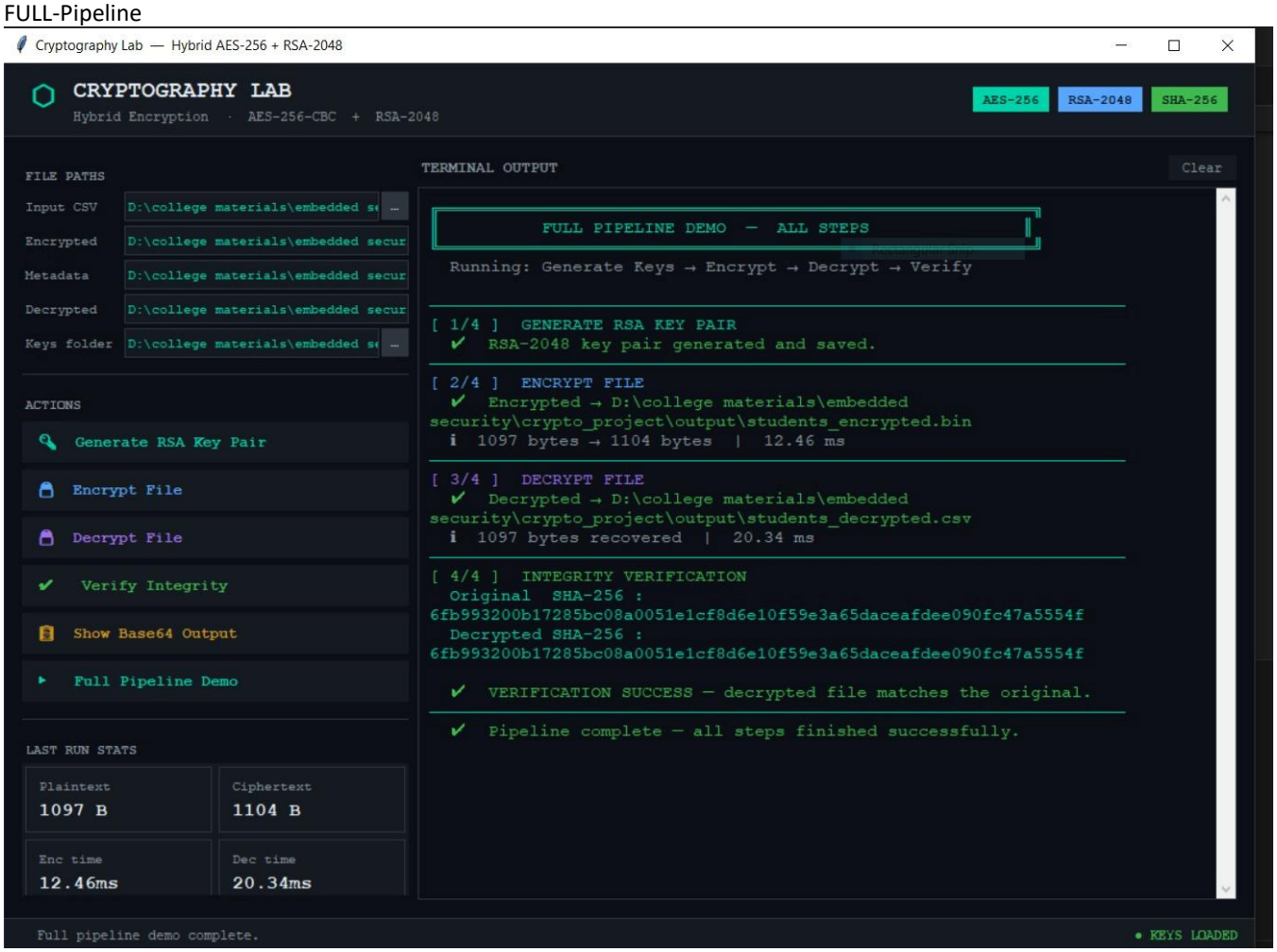
The RSA private key `keys/private_key.pem` is stored as a plain, unencrypted PEM file on disk. Any user or process with read access to this file can decrypt any ciphertext produced by this application. In a production system the private key would be protected with a passphrase using a Key Derivation Function (KDF), or stored in dedicated secure hardware such as a Hardware Security Module (HSM) or a Trusted Platform Module (TPM), making it impossible to extract even with physical access to the machine.

Screenshots :

GUI-Page:



FULL-Pipeline



Public-Key-RSA:

Private-Key-RSA:

private_key.pem M

keys > private_key.pem


```
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAqJZadbzRcHbAMxJ5/UYAIIYCBJoepyVMJPNFLt9pNVJVSTAA
OWeOMRKTT/QnaTLzDDr7lEt67SjdnPJ6BsSz5ZUH4cWWym+lQ3ACDpwQ0qllFuor
5CONIsNuJDpF9fesP3r2a8fza+AWPsJVNHdHkPvwW0OoBVpYWDrUR9ZYBUZhrZOX
HzTDVEEQs2Xalz+5ZUu3JNzke8/zzyq6E6GhEQx2rA/0Xuvgdbyog+jTu6zNBrDz
tgk0QCoBYOPlQX1NMnmwt9/2UvQCeoEl2ss93UfokO8rPhXU+WN5gQoTxMBCbauS
rXVdEbkstS45vHDbfXrx8pd4cCEWsxctwqCeVQIDAQABAoIBABRiY6s3Srq2vRwf
yIzJjUPheDZ+vSSYM6oHGS8m99Vo8GOCUJCgVxbTaGkDPE2xyaYJrrg6XTF2Xr92
qS8O0zK4D6Uqq0YsdDbggPsrvzP9o/m+lFKIQu04ZKWwPxvT1+mwEEItgd8iEAaE
IGHQ52AeOmyKlHMbPrHiYR8TiX6PlvjhwTlRlCRGBBN5WgVf/38ud8QpgVQ6kE0K
YbDaLh+gnyXV2dLdCZvtTnAFyHgf565ltgDG9GADWVGgvW8jiIiqqbHz4Z5HuACK
N6MbxFNLtfrA3pZ9hW18x7OrH1FqSVzfpYiz8ucgs6S3Ls3tKdkZc+l5kVh4DXG3
A32dXskCgYEAxKWyKZr4f4iyhB7CJzOE7vIkctV0LCaLDeATkAaK7gzVZPSp0sqD
xULQKBDiOq6pNPFYicw/s9lUd/bB3fUgwkPxLmpaRgNe2wkoZcSz+4eZT17D7FVb
8hk4A9RVWmqC0DdLleDYFsPXI/RKKg35skVI2cxfoWyqOPsRUspSLrkCgYEA23iO
Y+ECF1z3+emEeBvnqvhRxmbe2txLi03xWc7OnAVYkdGZXPmqeLWgziqrait9moNA
5fO9G3qnlkoLDphi6uPVaNmhY4O4trgY+xlAebqtWpSJhPwv2IV22PG5oHvb59lh
0udEbALEsyMv6jQAwEPry9x7Bptk6CUfCgsnPn0CgYB8vrgEmKBnFupX0cNMZuw/
QDBLAImPsGupw6nlRaFAXbfJMUPBymJI8pdls0AwBbilMyz9TSpkBswExe45xVSg
p8UJssHNhDnaLTcJR/z85QomRXF4LLYFIGZeV4ZFsyeEum8iht3NozKJRmD9yJvL
sRQDxhgsKOja+LoGkP4NUQKBgAUZqR1zFixa0bxyC80KwPh9KtFUhKtLYUoMdqZi
ovSHP0asO0co2ihR+DmyTid6eUHE+RYe8u8Xi5t3fwhIzEuvHb+BBzobzogYgSQc
sx3hE7Q+LW/0uFvsxhVGkMzE9BhQOfC/xRxJeC69i5iCut289lbXedhudgrbCfye
w7u1AoGBAKXlwGN/BrQV1pQLI4h76vceO586vPiKm4Ii7Wf44pWXJRnS9NSeIKbW
hZCUQaGcuCRSdpAL7cHNpz7tTe89Xa6LVe80syAVS06ly4ktA500garDSSSqNb5y
fZfxyFtrl35r/7CSHpb+cIQF5N/mi8VOePOtWNdiZKC+mSK818pF
-----END RSA PRIVATE KEY-----
```


Metadata-image:

metadata.json M

output > metadata.json > ...


```
{
  "iv": "1OfArmQO1EToQn5wEsaHaw==",
  "encrypted_aes_key": "CcmQAsHoiPwMddn8Vmsv8PReQ1K4kcuZo06tBXd8FlydP
  "aes_mode": "AES-256-CBC",
  "rsa_bits": 2048,
  "padding": "PKCS7"
}
```


Input-CSV:

students.csv

data > students.csv


```
StudentID,FirstName,LastName,Email,GPA,Major,Year
S001,Ahmed,Hassan,ahmed.hassan@university.edu,3.85,Computer Science,3
S002,Fatima,Ali,fatima.ali@university.edu,3.72,Electrical Engineering,2
S003,Omar,Khaled,omar.khaled@university.edu,3.91,Cybersecurity,4
S004,Nour,Ibrahim,nour.ibrahim@university.edu,3.45,Information Systems,1
S005,Youssef,Mahmoud,youssef.mahmoud@university.edu,3.67,Computer Science,2
S006,Layla,Ahmed,layla.ahmed@university.edu,3.88,Cybersecurity,3
S007,Karim,Mostafa,karim.mostafa@university.edu,3.55,Electrical Engineering,4
S008,Sara,Nasser,sara.nasser@university.edu,3.93,Computer Science,2
S009,Tarek,Farouk,tarek.farouk@university.edu,3.78,Information Systems,3
S010,Hana,Salem,hana.salem@university.edu,3.62,Cybersecurity,1
S011,Bilal,Younis,bilal.younis@university.edu,3.41,Computer Science,4
S012,Rania,Adel,rania.adel@university.edu,3.76,Electrical Engineering,2
S013,Ziad,Hamdan,ziad.hamdan@university.edu,3.83,Cybersecurity,3
S014,Dina,Fawzy,dina.fawzy@university.edu,3.59,Information Systems,1
S015,Malik,Osman,malik.osman@university.edu,3.97,Computer Science,4
```

Dec-CSV:

students_decrypted.csv

output > students_decrypted.csv

```
StudentID,FirstName,LastName,Email,GPA,Major,Year
S001,Ahmed,Hassan,ahmed.hassan@university.edu,3.85,Computer Science,3
S002,Fatima,Ali,fatima.ali@university.edu,3.72,Electrical Engineering,2
S003,Omar,Khaled,omar.khaled@university.edu,3.91,Cybersecurity,4
S004,Nour,Ibrahim,nour.ibrahim@university.edu,3.45,Information Systems,1
S005,Youssef,Mahmoud,youssef.mahmoud@university.edu,3.67,Computer Science,2
S006,Layla,Ahmed,layla.ahmed@university.edu,3.88,Cybersecurity,3
S007,Karim,Mostafa,karim.mostafa@university.edu,3.55,Electrical Engineering,4
S008,Sara,Nasser,sara.nasser@university.edu,3.93,Computer Science,2
S009,Tarek,Farouk,tarek.farouk@university.edu,3.78,Information Systems,3
S010,Hana,Salem,hana.salem@university.edu,3.62,Cybersecurity,1
S011,Bilal,Younis,bilal.younis@university.edu,3.41,Computer Science,4
S012,Rania,Adel,rania.adel@university.edu,3.76,Electrical Engineering,2
S013,Ziad,Hamdan,ziad.hamdan@university.edu,3.83,Cybersecurity,3
S014,Dina,Fawzy,dina.fawzy@university.edu,3.59,Information Systems,1
S015,Malik,Osman,malik.osman@university.edu,3.97,Computer Science,4
```

CONCLUSION

This project successfully demonstrates hybrid encryption in a realistic scenario. AES-256-CBC provides fast, secure symmetric encryption for the student records CSV, while RSA-2048 OAEP ensures the AES session key is never transmitted in plaintext. SHA-256 verification provides mathematical proof that decryption fully recovered the original data. The Tkinter GUI makes the entire pipeline accessible through a single Full Pipeline Demo button, producing clear terminal output at every step. The project meets all requirements of Path 3 and includes the bonus features of SHA-256 integrity checking, Base64 ciphertext display, and a complete graphical interface.